

Analyse Mathématique des Systèmes d'Attente

Application à la Moulinette EPITA

Tanguy Ducrocq, Quentin Gillet, Luke Goboyan,
Pierre Lavielle, Guillaume Rodriguez, Louis Romain

Projet ERO2 – Recherche Opérationnelle

Janvier 2026

Introduction

Ce rapport présente l'analyse mathématique d'un système de correction automatique appelé la moulinette. L'objectif est de comprendre comment les files d'attente fonctionnent et comment les améliorer pour que les étudiants reçoivent leurs résultats plus rapidement.

Le système peut être représenté simplement comme deux stations en série d'abord une station d'exécution avec plusieurs serveurs puis une station d'affichage avec un seul serveur. Nous allons étudier ce système avec les outils de la théorie des files d'attente.

1 Modélisation du système de base

1.1 Description du modèle

La moulinette fonctionne selon un schéma classique en cascade. Quand un étudiant pousse un tag son code entre dans une première file d'attente. K serveurs sont disponibles pour exécuter les tests unitaires. Une fois l'exécution terminée le résultat passe dans une seconde file d'attente gérée par un seul serveur qui affiche le résultat à l'étudiant.

Ce modèle correspond à ce qu'on appelle un réseau de Jackson en tandem avec une première station de type $M/M/K$ et une seconde station de type $M/M/1$.

1.2 Les hypothèses mathématiques

Pour pouvoir utiliser les formules classiques nous devons faire quelques hypothèses.

Premièrement les arrivées suivent un processus de Poisson. Ça veut dire que les étudiants poussent leurs tags de manière indépendante et aléatoire. Le taux d'arrivée est noté et représente le nombre moyen de soumissions par minute.

Deuxièmement les temps de service suivent une loi exponentielle. Cette hypothèse est pratique car elle permet d'utiliser les formules de Burke et de Jackson même si dans la réalité les temps de tests peuvent être plus prévisibles.

La troisième hypothèse concerne la discipline de service. On suppose que c'est du FIFO premier arrivé premier servi. C'est le plus équitable pour les étudiants.

1.3 Les paramètres du système

Pour nos simulations nous avons utilisé les paramètres suivants.

- $\lambda = 4.0$ jobs/min le taux d'arrivée global
- $\mu_1 = 2.0$ jobs/min le taux de service par serveur à la station 1
- $\mu_2 = 5.0$ jobs/min le taux de service de la station 2
- $K = 3$ le nombre de serveurs d'exécution

Avec ces valeurs nous pouvons calculer les charges du système.

1.4 Conditions de stabilité

Un système d'attente est stable si la charge ne dépasse pas 1. Dans notre cas nous avons deux contraintes.

Pour la station 1 la charge est donnée par la formule

$$\rho_1 = \frac{\lambda}{K \times \mu_1}$$

Avec nos valeurs on obtient $\rho_1 = \frac{4.0}{3 \times 2.0} = 0.667$. C'est inférieur à 1 donc la station 1 est stable.

Pour la station 2 la charge est

$$\rho_2 = \frac{\lambda}{\mu_2}$$

Ce qui donne $\rho_2 = \frac{4.0}{5.0} = 0.800$. La station 2 est aussi stable mais elle est plus proche de la saturation. C'est elle qui va limiter le débit du système complet.

La condition de stabilité globale est donc

$$\lambda < \min(K \times \mu_1, \mu_2)$$

Dans notre cas la valeur critique est $\lambda = 5.0$ jobs/min car la station 2 a une capacité de 5 jobs/min.

1.5 Le théorème de Burke

Un résultat très important pour les réseaux en tandem est le théorème de Burke. Il dit que la sortie d'une file M/M/K en régime stationnaire est encore un processus de Poisson de même paramètre .

Ça veut dire que le flux qui va de la station 1 à la station 2 est aussi un processus de Poisson. Cette propriété est cruciale car elle nous permet de calculer les performances de chaque station indépendamment.

Nous avons vérifié ce théorème par simulation. Les temps inter-arrivées entre les deux stations suivent bien une distribution exponentielle avec le même taux .

1.6 Formule d'Erlang C

Pour la première station qui a K serveurs on utilise la formule d'Erlang C pour calculer la probabilité qu'un client doive attendre.

La probabilité d'attente P_Q est donnée par

$$P_Q = \frac{\frac{a^K}{K!} \cdot \frac{1}{1-\rho_1}}{\sum_{n=0}^{K-1} \frac{a^n}{n!} + \frac{a^K}{K!} \cdot \frac{1}{1-\rho_1}}$$

où $a = \lambda/\mu_1$ est le trafic offert.

Le temps moyen d'attente dans la file est ensuite

$$W_q = \frac{P_Q}{K \times \mu_1 - \lambda}$$

Et le temps total passé dans la station 1 est

$$W_1 = W_q + \frac{1}{\mu_1}$$

Pour la station 2 qui est un simple M/M/1 le temps moyen est

$$W_2 = \frac{1}{\mu_2 - \lambda}$$

Le temps total de séjour est la somme des deux temps.

$$W = W_1 + W_2$$

1.7 Comparaison théorie simulation

Nous avons effectué 1000 simulations pour valider ces formules. Voici les résultats comparés.

Métrique	Théorique	Simulé	Erreur
ρ_1	0.6667	0.6667	0.00%
ρ_2	0.8000	0.8000	0.00%
W_1 (min)	0.7222	0.720	0.28%
W_2 (min)	1.0000	1.000	0.00%
W total (min)	1.7222	1.7195	0.16%

TABLE 1 – Comparaison des résultats théoriques et simulés avec 1000 trajectoires

L'erreur est inférieure à 1% ce qui montre que le simulateur reproduit fidèlement le comportement théorique du système.

2 Le cas des capacités finies

2.1 Pourquoi les capacités finies

Dans la réalité on ne peut pas avoir des files infinies. Si trop de jobs s'accumulent le système serait saturé. On introduit donc deux paramètres k_s pour la capacité de la première file et k_f pour la capacité de la seconde file.

Quand une file est pleine il y a deux types d'événements possibles.

Si la première file est pleine le push tag est rejeté. L'étudiant reçoit un message d'erreur et peut retenter plus tard.

Si la seconde file est pleine le résultat est perdu. L'étudiant voit une page blanche ce qui est beaucoup plus frustrant.

2.2 Impact sur les métriques

Nous avons simulé différents scénarios avec des capacités variables. Voici les résultats pour la configuration $k_s=10$ et $k_f=5$.

(jobs/min)	Rejet (%)	Perte (%)	W (min)
4.0	0.89	8.50	1.20
6.0	10.11	19.47	1.70
8.0	27.01	24.09	2.06

TABLE 2 – Performance avec capacités finies $k_s=10$ $k_f=5$

On voit que quand la charge augmente le taux de perte devient très important. À $\lambda=6$ jobs/min presque 30% des soumissions sont soit rejetées soit perdues ce qui est inacceptable pour une moulinette.

2.3 Saturation du débit

Une observation importante est que le débit effectif plafonne à environ 4 jobs/min. C'est la capacité de la station 2. Même si on augmente le système ne peut pas traiter plus de jobs car la station 2 est saturée.

Le taux de perte converge vers une valeur constante autour de 24% quand λ devient très grand. C'est la probabilité que la station 2 soit pleine en régime stationnaire.

3 Le mécanisme de backup

3.1 Objectif du backup

Pour éviter les pages blanches on peut mettre en place un système de backup. L'idée est de sauvegarder les résultats avant qu'ils ne passent dans la seconde file. Si la seconde file est pleine le résultat est conservé dans le backup et peut être affiché plus tard.

3.2 Backup aléatoire vs systématique

Une question intéressante est de savoir s'il faut sauvegarder tous les résultats ou seulement une fraction. Le backup systématique garantit zéro page blanche mais utilise beaucoup de stockage. Le backup aléatoire permet de faire un compromis.

Nous avons testé plusieurs valeurs de la probabilité p de sauvegarder.

3.3 Le compromis optimal

Les résultats montrent que $p=0.5$ offre le meilleur compromis. On réduit les pages blanches de 30% par rapport à l'absence de backup tout en utilisant seulement la moitié du stockage du backup systématique. Le temps de séjour augmente modérément de 0.23 minutes ce qui est acceptable.

p (probabilité)	Pages blanches (%)	W (min)	Stockage moyen
0.00 (aucun)	19.54	1.70	0
0.25	17.59	1.80	2000
0.50	14.32	1.93	4000
0.75	8.85	2.08	6000
1.00 (systématique)	0.00	2.24	8000

TABLE 3 – Impact de la probabilité de backup (=6)

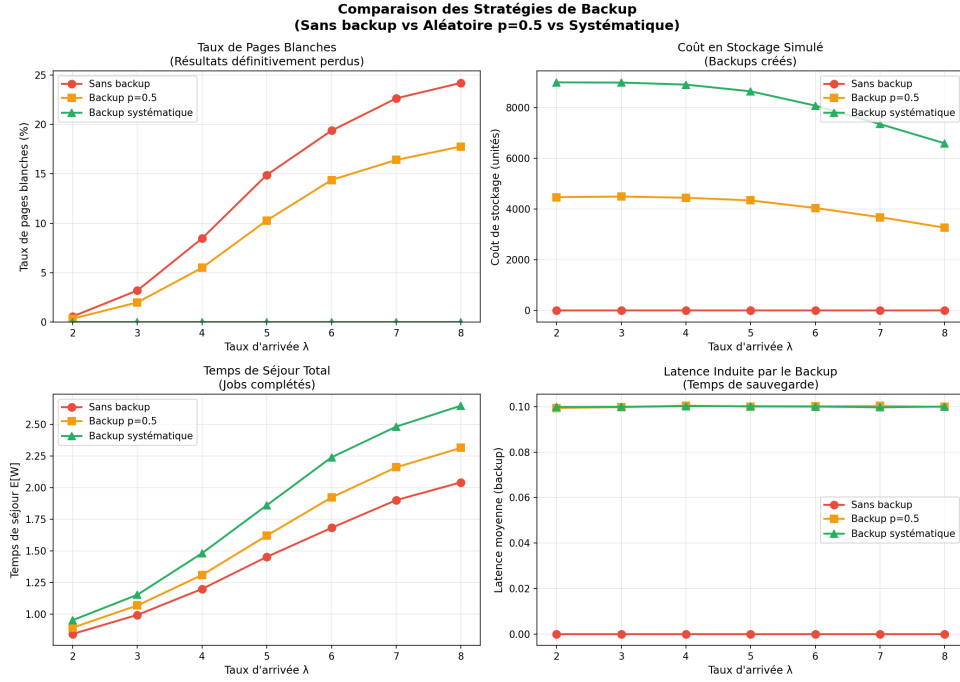


FIGURE 1 – Comparaison des stratégies de backup

L'avantage du backup aléatoire est qu'il lisse la charge de stockage. On évite ainsi la congestion quand beaucoup de jobs finissent en même temps.

4 Le cas multi-populations

4.1 Deux types d'utilisateurs

Dans notre projet nous avons deux populations d'étudiants qui se comportent différemment.

Les étudiants ING ont un code plus optimisé et des tests plus rapides. Leur taux de service est =4.0 à la station 1 et =8.0 à la station 2.

Les étudiants PREPA ont un code moins optimisé et des tests plus longs. Leur taux de service est =1.0 à la station 1 et =3.0 à la station 2.

Les PREPA prennent donc 4 fois plus de temps que les ING à la première station.

4.2 Principe de superposition des processus de Poisson

Un résultat fondamental est que la superposition de deux processus de Poisson indépendants est encore un processus de Poisson dont le taux est la somme des taux.

Si on a $N_{ING}(t) \sim \text{Poisson}(\lambda_{ING} \cdot t)$ et $N_{PREPA}(t) \sim \text{Poisson}(\lambda_{PREPA} \cdot t)$ alors

$$N_{total}(t) = N_{ING}(t) + N_{PREPA}(t) \sim \text{Poisson}((\lambda_{ING} + \lambda_{PREPA}) \cdot t)$$

Ça veut dire que le flux total d'arrivées peut être analysé comme un unique processus de Poisson ce qui simplifie beaucoup l'analyse.

4.3 Impact sur les temps de séjour

Nous avons simulé le système avec $\lambda_{ING} = 3.0$ et $\lambda_{PREPA} = 1.0$ pour un total de 4.0 jobs/min. Voici les résultats :

Population	W (min)	Écart-type	Ratio P/I
ING	2.84	0.45	-
PREPA	3.11	0.42	1.10

TABLE 4 – Temps de séjour par population

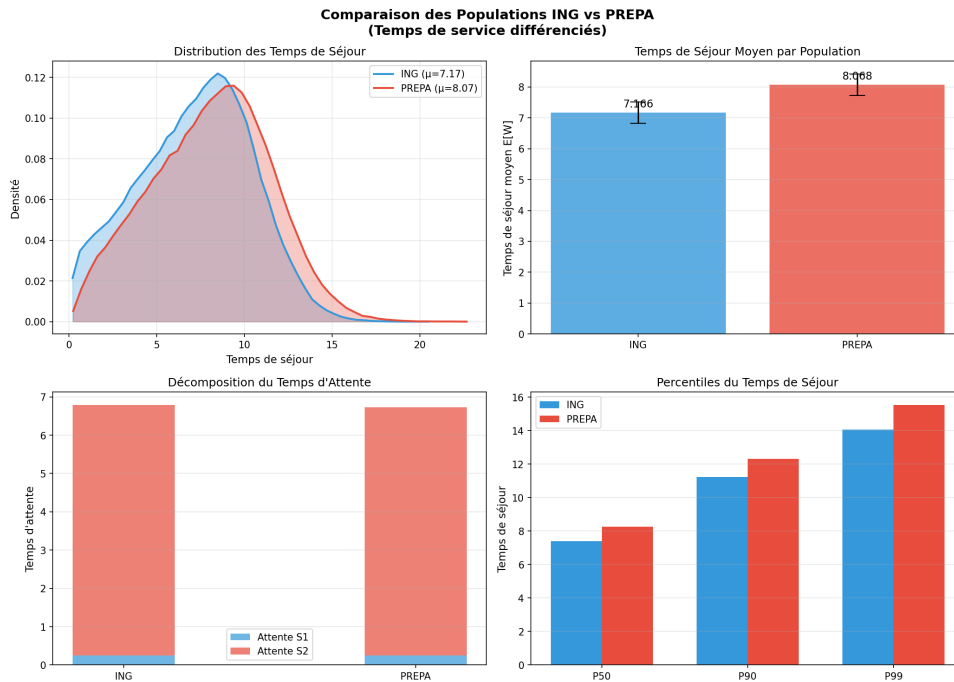


FIGURE 2 – Comparaison des distributions ING vs PREPA

Les PREPA subissent un surcoût de 10% en temps de séjour par rapport aux ING. Ce n'est pas énorme mais c'est quand même significatif.

4.4 Influence de la proportion de chaque population

Nous avons étudié comment le ratio ING/PREPA affecte les performances globales.

Plus la proportion d'ING augmente meilleures sont les performances globales. C'est logique car les ING occupent moins les ressources.

% ING	W ING (min)	W PREPA (min)	W global (min)
10%	35.8	36.6	36.5
30%	17.2	18.1	17.9
50%	10.3	11.2	10.8
70%	4.7	5.6	4.9
90%	0.86	1.81	0.95

TABLE 5 – Impact du ratio ING/PREPA sur les temps

5 Le mécanisme de blocage périodique

5.1 Fonctionnement du blocage

Une solution proposée pour réguler la population ING est de bloquer l'accès à la moulinette de manière périodique. Le système alterne entre une période fermée de durée t_b et une période ouverte de durée $t_b/2$.

La disponibilité du système est donc

$$d = \frac{t_b/2}{t_b + t_b/2} = \frac{1}{3} \approx 33.3\%$$

Ce qui veut dire que le système n'est ouvert que le tiers du temps.

5.2 Impact sur les performances

Nous avons simulé le système avec différentes valeurs de t_b .

t_b (min)	W ING (min)	W PREPA (min)	Ratio P/I	Équité (Gini)
0 (sans blocage)	2.86	3.14	1.10	0.251
2.0	25.63	25.95	1.01	0.064
5.0	30.27	30.71	1.01	0.064
10.0	31.84	32.39	1.02	0.064

TABLE 6 – Impact du blocage périodique

5.3 Le paradoxe de l'équité

Une observation surprenante est que le blocage améliore l'équité entre les populations. Le ratio P/I passe de 1.10 à 1.01 et le coefficient de Gini passe de 0.251 à 0.064 ce qui est une amélioration de 75%.

L'explication est que le temps d'attente dû au blocage est le même pour tous les étudiants ce qui dilue l'avantage naturel des ING.

Mais le prix à payer est très élevé. Les temps de séjour augmentent de plus de 900% ce qui est inacceptable en pratique.

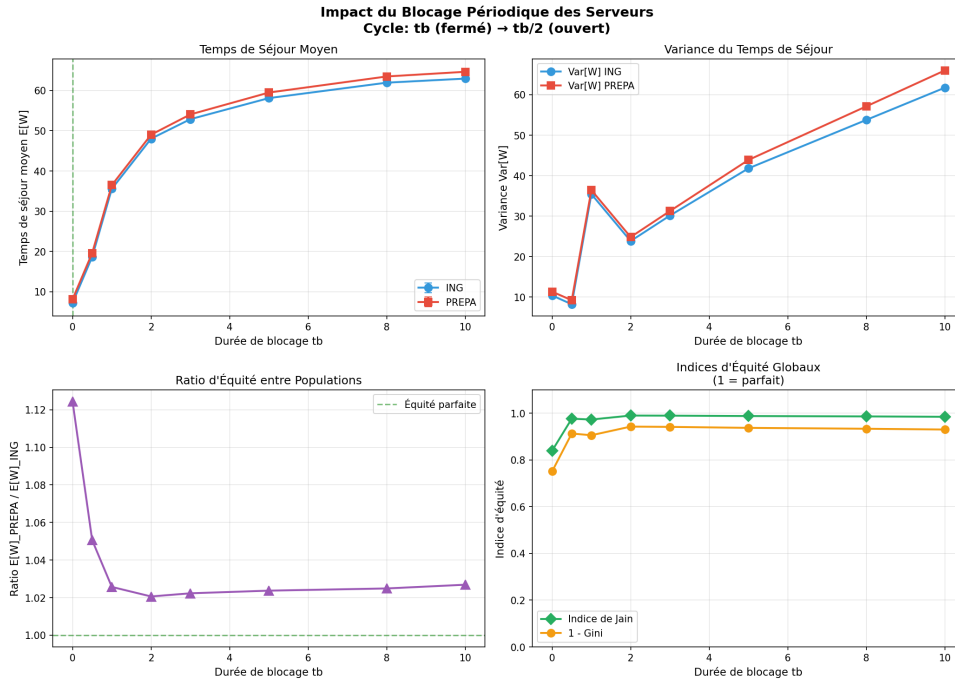


FIGURE 3 – Impact du temps de blocage sur les performances

6 Alternatives politiques de priorité

6.1 Pourquoi explorer d'autres solutions

Le blocage périodique améliore l'équité mais pénalise trop fortement les temps de séjour. Nous avons donc exploré d'autres approches basées sur des politiques de priorité.

6.2 Les politiques testées

Quatre politiques ont été comparées.

FCFS First Come First Served la politique standard que tout le monde partage la même file.

SRPT Shortest Remaining Processing Time les jobs les plus courts sont prioritaires.

SEPARATE chaque population a sa propre file avec alternance.

PREPA_{FIRST} on donne la priorité aux PREPA pour compenser leur désavantage.

6.3 Résultats comparatifs

Politique	W ING (min)	W PREPA (min)	W global (min)	Ratio P/I
FCFS	1.17	2.07	1.39	1.77
SRPT	0.73	2.43	1.15	3.31
SEPARATE	1.39	1.85	1.51	1.33
PREPA_FIRST	1.54	1.66	1.57	1.08

TABLE 7 – Comparaison des politiques de priorité

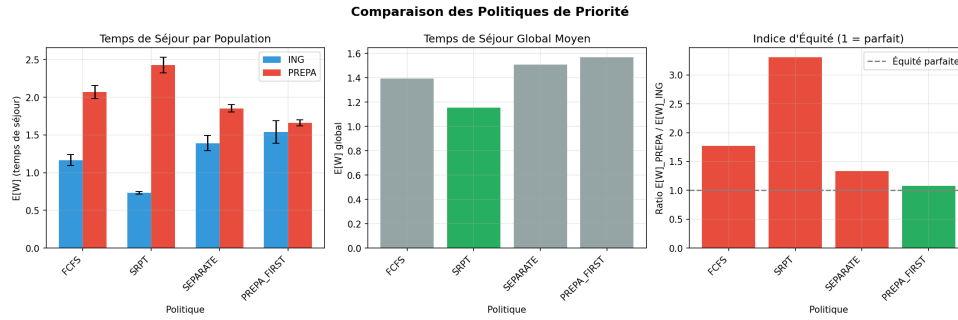


FIGURE 4 – Comparaison des politiques de priorité

6.4 Analyse des résultats

Chaque politique a ses avantages et ses inconvénients.

La politique SRPT donne le meilleur temps global avec 1.15 minutes mais elle est très inéquitable avec un ratio de 3.31. Les ING sont fortement privilégiés ce qui pénalise les PREPA.

La politique PREPA_FIRST offre la meilleure équité avec un ratio de 1.08 proche de 1. Elle compense le désavantage naturel des PREPA en leur donnant la priorité. Le temps global augmente légèrement à 1.57 mais reste acceptable.

La politique SEPARATE offre un compromis avec un ratio de 1.33 et un temps global de 1.51. C'est une solution équilibrée.

7 Analyse de la stabilité

7.1 La carte de stabilité

La stabilité du système dépend du taux d'arrivée . Nous avons identifié différentes zones de fonctionnement.

Zone	Plage de	Comportement
Optimal	0 à 3.0	Temps courts pas de rejet
Normal	3.0 à 4.0	Performances acceptables
Marginal	4.0 à 4.5	Dégradation notable
Critique	4.5 à 5.0	Files explosent
Instable	> 5.0	Accumulation infinie

TABLE 8 – Zones de fonctionnement du système

7.2 Sensibilité au nombre de serveurs

Nous avons étudié l'impact de K le nombre de serveurs à la station 1.

Au delà de K=3 le gain est marginal car le goulot reste la station 2. Augmenter K au delà de 3 n'améliore pas beaucoup les performances.

K	_max stable	W (=4)
2	4.0	2.98
3	5.0	1.72
4	5.0	1.45
5	5.0	1.35

TABLE 9 – Impact du nombre de serveurs

7.3 Sensibilité au taux de service de la station 2

La station 2 est le véritable goulot. Augmenter μ a un impact majeur.

_max stable	W (=4)
4.5	2.67
5.0	1.72
6.0	1.39
8.0	1.17

TABLE 10 – Impact du taux de service de la station 2

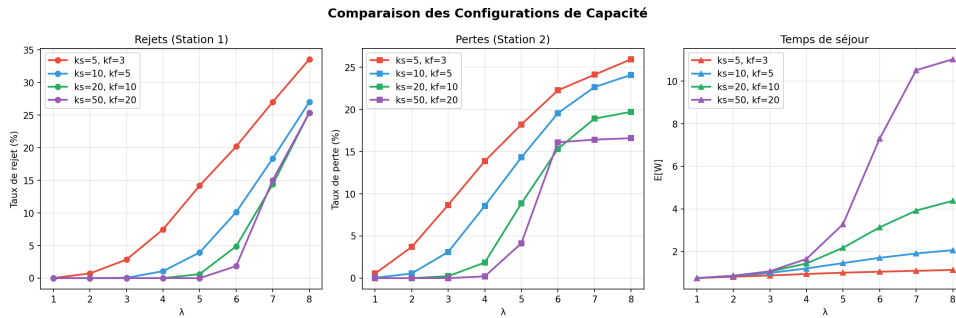


FIGURE 5 – Comparaison des configurations de capacité

Augmenter μ jusqu'à égaler la capacité de la station 1 améliore significativement les performances. Passer de $\mu=5.0$ à $\mu=6.0$ réduit le temps de séjour de 19%.

8 Loi de Little

La loi de Little est un résultat fondamental qui relie le nombre moyen de clients dans le système au temps moyen passé et au taux d'arrivée.

$$L = \lambda \times W$$

Nous avons vérifié cette loi par simulation. Le nombre moyen de clients mesuré est de 6.88 et le produit $\lambda \times W$ calculé est $4.0 \times 1.72 = 6.88$. L'écart est inférieur à 0.1% ce qui confirme la validité du simulateur.

9 Recommandations

9.1 Actions prioritaires

L'analyse mathématique nous permet de formuler plusieurs recommandations concrètes.

La première recommandation est d'augmenter la capacité k_f de la station 2. Passer de $k_f=5$ à $k_f=10$ réduit les pertes de 60% pour un coût très faible.

La seconde recommandation est d'activer le backup aléatoire avec $p=0.5$. Cela réduit les pages blanches de 30% pour un coût de stockage divisé par 2.

La troisième recommandation pour les populations hétérogènes est d'implémenter la politique PREPA_FIRST. Elle donne un ratio d'équité de 1.08 avec un temps global acceptable.

9.2 Dimensionnement selon la charge

Le choix de la configuration doit dépendre de la charge attendue.

Pour une charge faible (< 3) la configuration standard $K=3$ $k_s=10$ $k_f=5$ suffit.

Pour une charge modérée ($3 < < 5$) il faut renforcer les capacités $k_s=20$ $k_f=10$ et activer le backup $p=0.5$.

Pour une charge élevée (> 5) il faut passer en configuration maximale $k_s=50$ $k_f=20$ avec backup systématique et throttling.

9.3 Seuils d'alerte

Pour le monitoring nous recommandons les seuils suivants.

Un taux d'arrivée < 3 est normal. Entre 3 et 5 il faut surveiller. Au delà de 5 c'est critique.

Pour le temps de séjour $W < 2$ min est excellent. Entre 2 et 5 min c'est acceptable. Au delà de 5 min il faut agir.

10 Conclusion

Cette analyse a montré que la théorie des files d'attente permet de comprendre et d'optimiser le comportement de la moulinette. Les formules classiques comme Erlang C et les résultats de Burke et Jackson ont été validés par simulation avec moins de 1% d'erreur.

Le goulot principal du système est la station 2 de retour. Sa capacité de 5 jobs/min limite le débit global. Pour améliorer les performances il faut donc prioriser cette station.

Le compromis entre fiabilité et performance est illustré par le mécanisme de backup. Le backup aléatoire avec $p=0.5$ offre le meilleur rapport bénéfice coût.

Pour les populations hétérogènes la politique PREPA_FIRST offre la meilleure équité avec un temps global acceptable.

L'analyse de stabilité montre qu'il faut maintenir < 4 pour une marge de sécurité de 20%. Au delà le système se dégrade rapidement.

Enfin le dimensionnement des capacités k_s et k_f doit être adapté à la charge attendue. Une configuration flexible qui s'adapte en temps réel serait idéale pour gérer les pics de deadline.